

Apunte 29 — Fase 6: generalizar no es entrenar (y el motor multi-topic)

Proyecto Froth · aprender Machine Learning construyendo

1 La aclaración que abre la fase

El plan de Batman: “iterar cada uno de los 22 títulos del cohorte y seguir entrenando”. Correcto en espíritu — pero son **dos actos distintos**:

Concepto: generalizar (6.1–6.3) vs. entrenar (6.5)

- **Iterar los 22 títulos NO entrena nada**: el modelo (SPECTER) no cambia ni un peso. Lo que se endurece es el **código** — cada título raro que rompe algo obliga a arreglarlo. Es la escalera 1→2→22 del plan original.
- **Entrenar de verdad** (6.5) sí cambia los pesos del modelo: fine-tuning de SPECTER con los miles de abstracts acumulados de los 22 corpus. PyTorch, loss, épocas — en la GPU de Batman (RTX 5060, confirmada capaz).

Los dos actos se alimentan: la generalización produce el dataset del entrenamiento.

2 El motor multi-topic (6.1)

Hasta ayer el topic vivía *clavado* en `config.py`. Ahora:

Concepto: `config.set_topic(título)`

Una llamada re-apunta TODO el motor: deriva términos de búsqueda nuevos y mueve las rutas de datos a una carpeta propia (`2_Datos/topics/<slug>/`). Funciona porque cada módulo lee `config.*` en el momento de ejecutar, no al importar — un detalle de diseño que pagó dividendos. (Trampa clásica cazada: `def f(x=config.TOPIC)` congela el valor al importar; se corrigió a `x=None` + resolver adentro.)

Concepto: `derive_search_terms()`: de título a búsquedas

Los términos del topic 1 se escribieron a mano. Para CUALQUIER título, la heurística de `froth/terms.py`: quita palabras de relleno (“innovative approaches for the..”), conserva **frases reales del título** (bigramas/trigramas consecutivos: *froth flotation*, *scheelite ores*, *Quantitative XRD*), respeta **acrónimos** (XRD, LIBS, NiMH) y extrae el **contenido de paréntesis** (ahí viven los elementos y yacimientos). Determinista, sin dependencias. La capa LLM local (Ollama) vendrá encima como refinador opcional — decisión ya tomada: local, privado, gratis.

3 El hito 1→2 (probado)

Pipeline completo sobre “*scheelite ores by froth flotation*” **sin tocar código**: 47 papers, carpeta propia, y clusters con sentido metalúrgico (concentrador Falcon, reactivos SNF, espumantes). Y la prueba pagó de inmediato:

Ojo / error típico

Lo que el 2º topic enseñó (por esto existe la escalera):

1. La keyword “nbsp” delató que los abstracts de Crossref traen **entidades HTML** () — el limpiador ahora hace `html.unescape`.
2. El `.gitignore` no cubría las carpetas nuevas por topic — los datos casi acaban en GitHub.
3. En corpus chicos (47 papers) la granularidad fija de 8 deja demasiado ruido → el pipeline ahora acepta `min_cluster_size="auto"` (elige por DBCV, Apunte 27, por corpus).

Tres arreglos reales con UN solo título nuevo. Los 22 del batch encontrarán más — ese es exactamente su trabajo.

4 El desfile (6.2)

`froth/batch.py` lee los 22 títulos del CSV del cohorte y corre el pipeline para cada uno, con reporte de robustez por topic (papers, relevantes, clusters, ruido, granularidad elegida, segundos, estado). Diseño defensivo: el reporte se guarda tras **cada** topic y las corridas siguientes **reanudan** donde quedaron (un rate-limit no pierde nada). Los fallos no detienen el desfile: quedan como FAILED en la tabla — y esa tabla es la agenda de arreglos de la 6.3.

En una frase

Iterar títulos endurece el código (generalizar); cambiar pesos es entrenar (6.5). El motor ya es multi-topic (`set_topic` + términos derivados + carpetas por topic), probado con scheelite ores — que de paso regaló 3 arreglos. El batch de 22 corre con reanudación y reporte por topic: su tabla de fallos es el plan de trabajo de la 6.3.